

Multilingual & Multimodal Models

Dr. Sreerama K. Murthy

Today's topics

- Multilingual models
- Image Generation – Very brief intro to diffusion models
- CLIP
- Vision Transformers
- Multimodal models
- Architecture & Enterprise patterns
- Evaluation

Beyond English

- Problem: Most training data is English. Most of the world is not.
- The old way (Pipeline):
 - *Translate --> Process (in English) --> Translate back.*
 - Latency high, error propagation high
- The New way (Native Multilingualism): One model, many languages. The model “thinks” in concepts, not specific English words.
- The Goal: Cross-lingual generalization (Train on English, infer on Tamil)

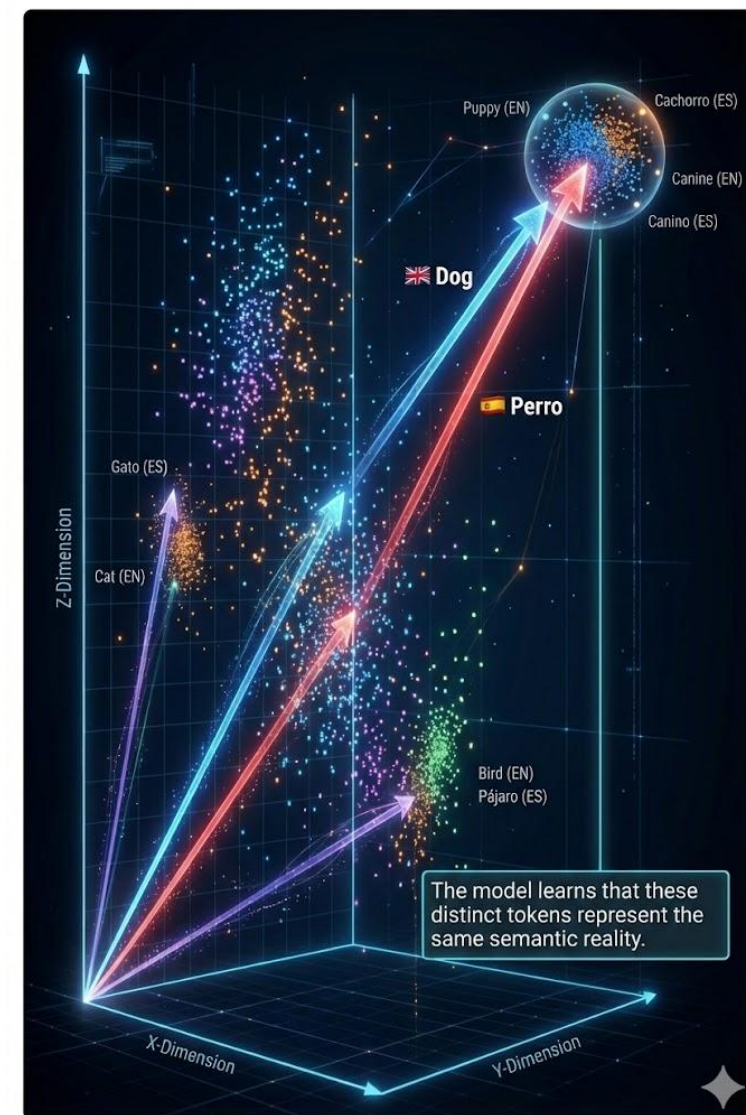


Why Multilingual Modelling Works

- ✓ Most languages share deep patterns: subjects, verbs, objects, etc
- ✓ Sub-word level tokenization allows shared representation of concepts
- ✓ Even unrelated scripts partially overlap in semantic space
- ✓ This is why a model trained on English + French often works on Spanish

Shared Embedding Spaces

- **Unified Vocabulary:** Instead of separate vocabs, we use a massive shared vocabulary (often 250k + tokens)
- **Alignment:** During pre-training (Masked Language Modelling), the model sees parallel text (or enough mixed text) to align the vector spaces of both languages.
- **Result:** A “semantic universal language” inside the model’s weights



```
from transformers import AutoTokenizer

# Load a popular multilingual tokenizer (XLM-RoBERTa)
# Vocabulary size: ~250,000 tokens
tokenizer = AutoTokenizer.from_pretrained('xlm-roberta-base')
```

```
print(f"Vocab Size: {tokenizer.vocab_size}")
```

```
# 1. THE SHARED ANCHOR EFFECT
```

```
# Notice how 'Paris' gets the SAME Token ID (5676), regardless of the sentence lang
# This acts as the mathematical 'anchor' we discussed.
```

```
en_text = "I love Paris."
```

```
fr_text = "J'adore Paris."
```

```
print(f"\nEnglish: {tokenizer.encode(en_text)}")
```

```
print(f"French: {tokenizer.encode(fr_text)}")
```

```
# Output English: [0, 87, 5155, 5676, 5, 2] <- See 5676
```

```
# Output French: [0, 688, 705, 30720, 5676, 5, 2] <- See 5676
```

```
# 2. SCRIPT AGNOSTICISM
```

```
# The tokenizer handles Hindi, Chinese, and English using the same lookup table.
```

```
# It doesn't switch modes.
```

```
mixed_text = "Hello नमस्ते 你好"
```

```
tokens = tokenizer.tokenize(mixed_text)
```

```
ids = tokenizer.encode(mixed_text)
```

```
print(f"\nOriginal: {mixed_text}")
```

```
print(f"Tokens: {tokens}")
```

```
print(f"IDs: {ids}")
```

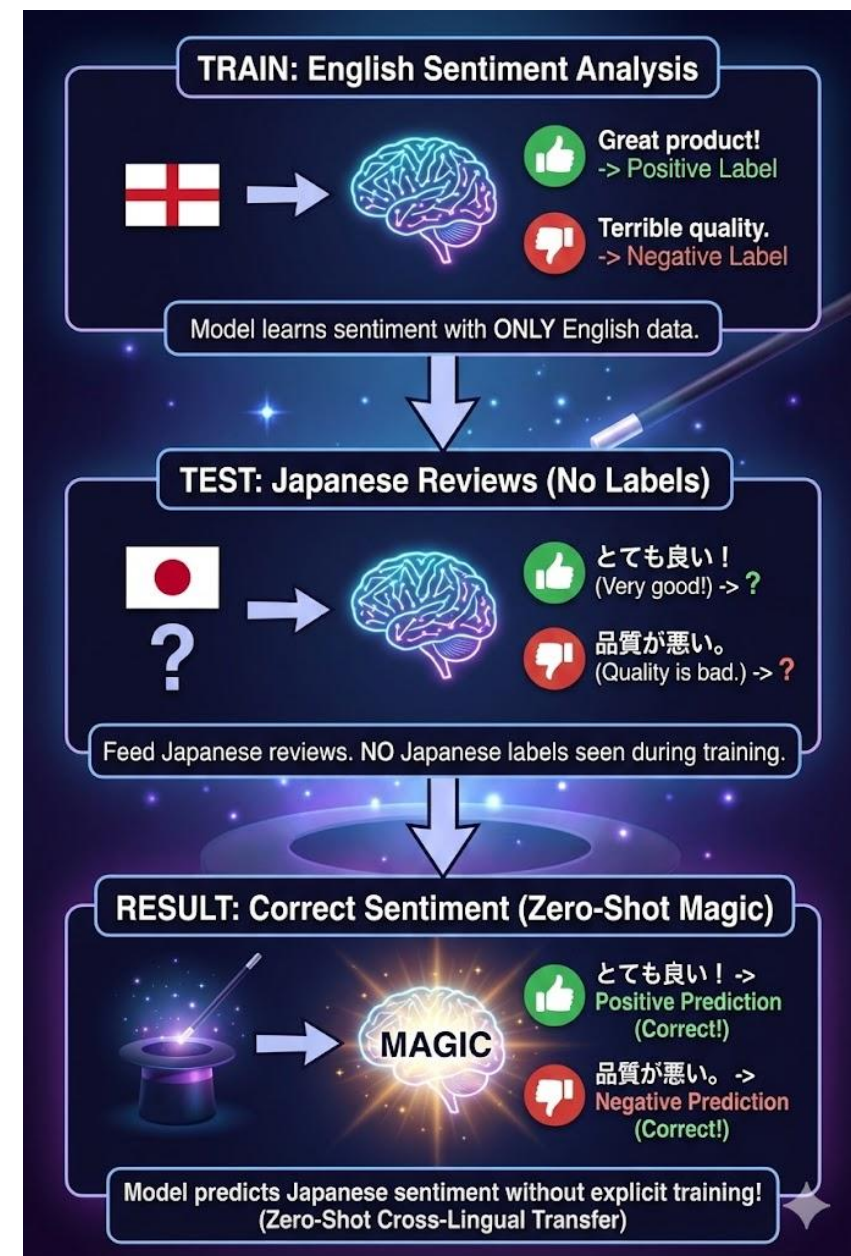
Tokenization at Scale: Handling the script barrier

- Byte-pair Encoding (BPE) & Sentence Piece: The standard for multilingual models
- Scripts with shared roots → more shared tokens (Latin, Cyrillic)
- For unrelated scripts, break into subword units and share morphemes.
- Large vocab (~ 250k tokens), supports dozens of languages efficiently.
- Caution: If you domain uses a niche language(eg: Tulu), stock tokenizers might fragment words too aggressively, destroying semantic meaning



Zero-Shot Cross-Lingual Transfer

- Definition: Fine-tuning a model on a downstream task in a source language (usually English) and evaluating it on a target language without seeing any labelled data in the target language
- Why this matters for ROI: You don't need to pay for labelling data in 50 languages. Label once (English), deploy everywhere
- Limitations:
 - Performance drops on “culturally specific” nuances (idioms, sarcasm).
 - As you add more languages to a fixed-model, performance on all languages eventually drops



The Hero Models

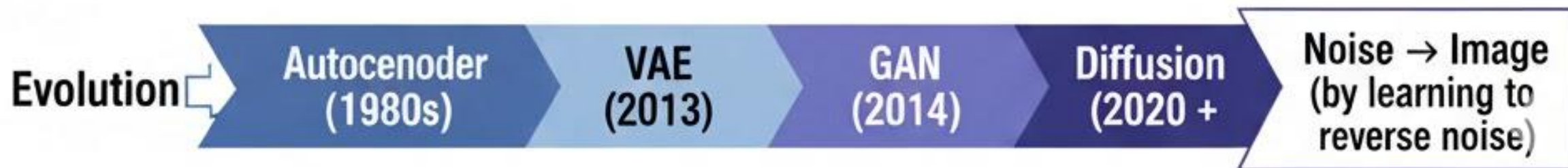
- **Encoders (Understanding):**
 - mBert: The grandfather. Good for classification
 - XLM-R (RoBERTa): Trained on 100 languages (CommonCrawl). The standard for non-generative tasks
- **Encoder-Decoder (Translation):**
 - NLLB (No Language Left Behind): Meta's models capable of 200+ languages
- **Decoders (Generation):**
 - BLOOM: The first massive open multilingual LLM.
 - GPT-4/Llama 3: Implicitly multilingual due to massive webscrape

From Languages to Images...

We have spent time so far, understanding text generation..

Let us now try and understand the image generation process..

A brief history of image generation models:



The core idea

You ask an experienced artist to help clean up an image painted by someone else – but has been corrupted by noise.

Tell them that this is a “Painting of an alien playing a guitar in the style of H.R. Giger”. Ask them to use this info and clean up.



Every block of stone has a
statue inside it and it is the
task of the sculptor to
discover it.

MICHELANGELO

Like a Michelangelo, a
diffusion model “sees”
the painting in a noisy
palette and goes ahead
removing the noise
step-by-step until the
full picture is visible!

"Removing Noise" over 50 steps



1



20



30



40



50

Diffusion Model: Behind the scenes

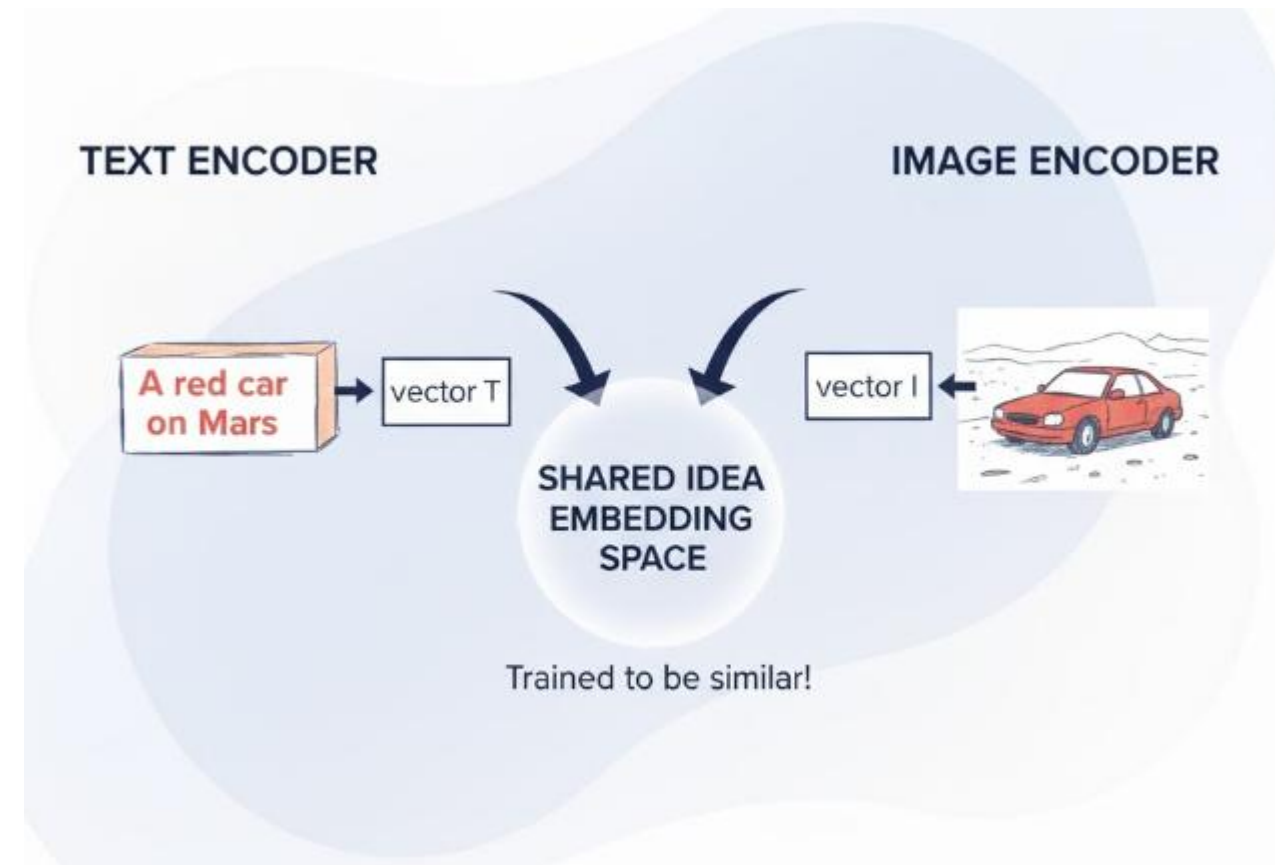
- Diffusion doesn't memorize pictures - it memorizes the *process of cleaning them*.
- Diffusion models have two phases: Forward phase where noise is added and the reverse phase where noise is removed
- During training the model learns to predict *just the noise* added at each step. Much like a restorer who first damages a photo on purpose and then masters the art of repairing it.
- Each step slightly cleans the image, guided by what the model *thinks* should be there

How does Diffusion relate the image to the caption provided?

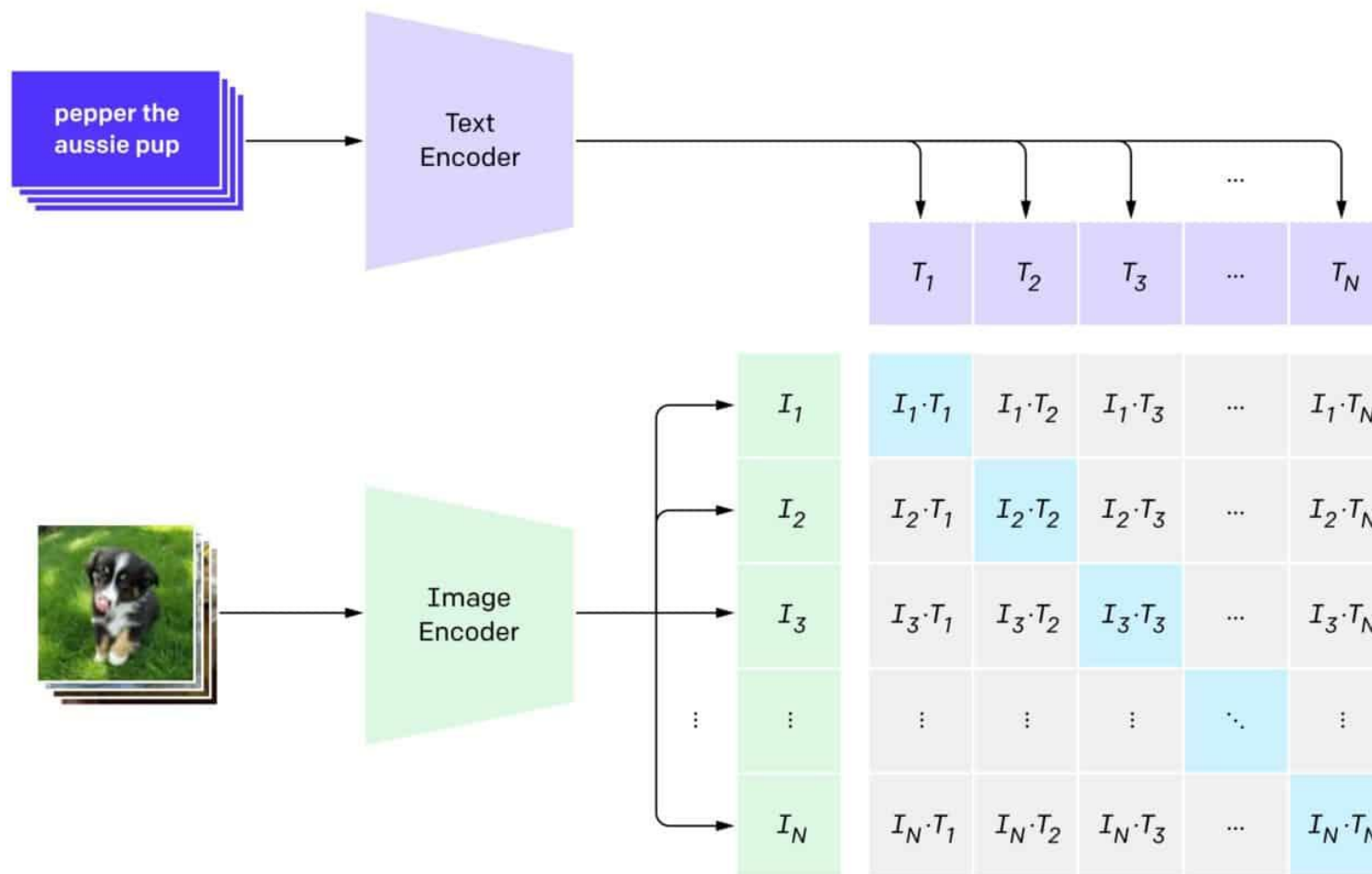
That's where CLIP comes in...

CLIP: Contrastive Language-Image Pre-Training

- Developed by OpenAI in 2021 using 400M image-caption pairs
- Text encoder + Image encoder trained together
- Goal: Map both words and pictures into the same contextual space



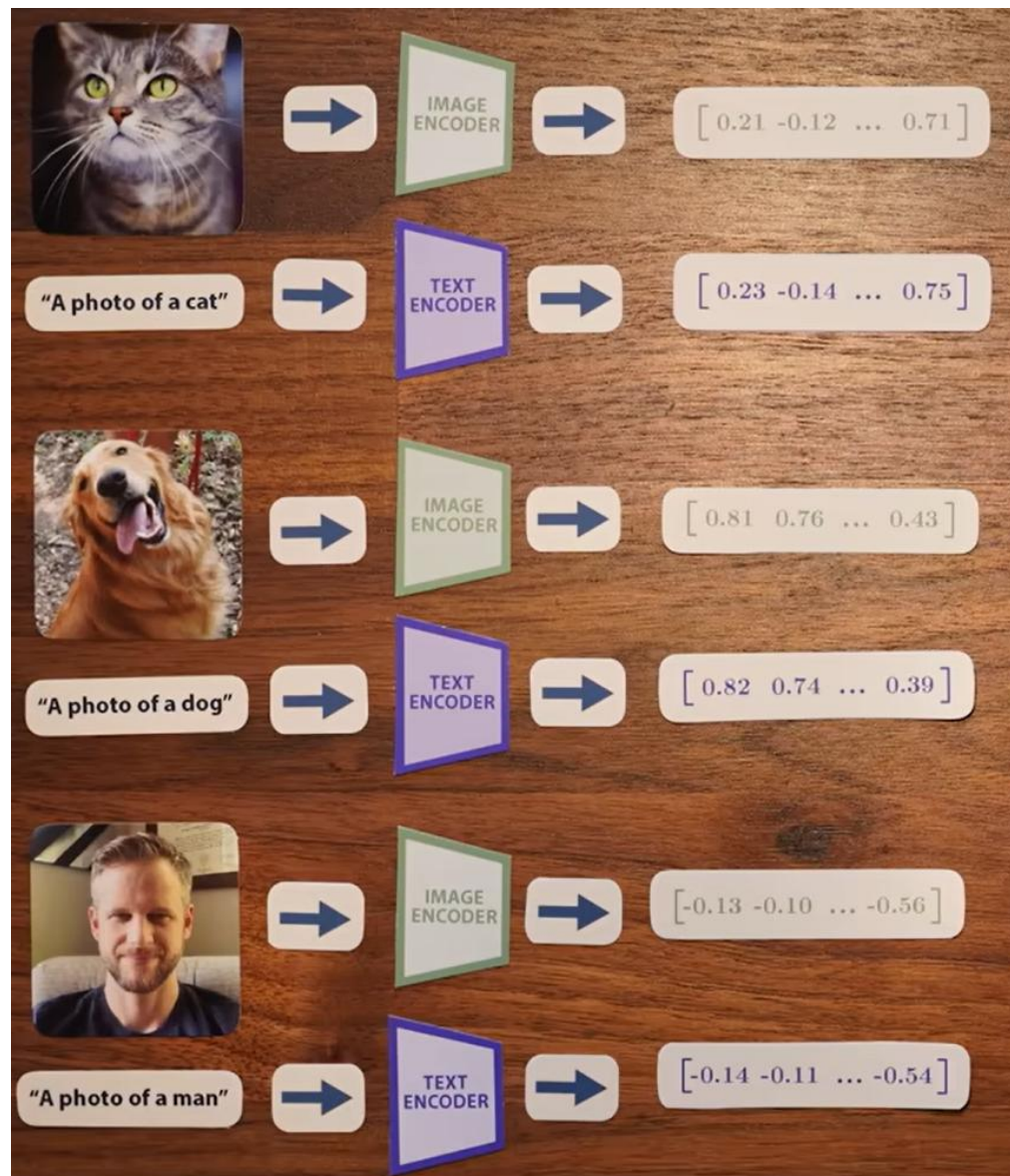
CLIP training



CLIP

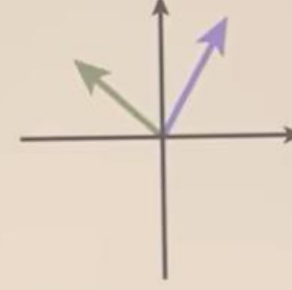
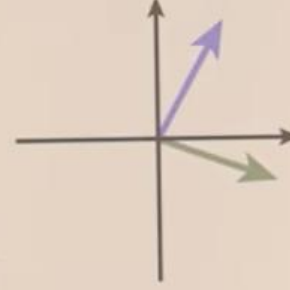
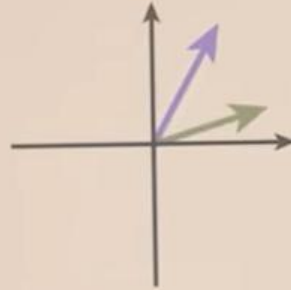
- Learns by *contrasting* correct and incorrect pairs.
- Matching pairs → closer in space.
- Non-matching pairs → pushed apart.
- Outcome: related text & image vectors point in same direction.

CLIP training

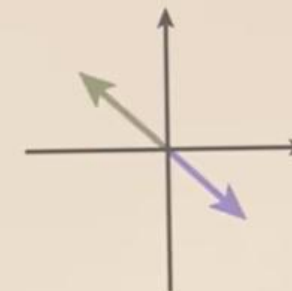
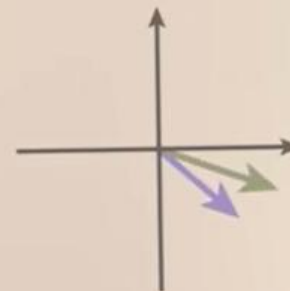
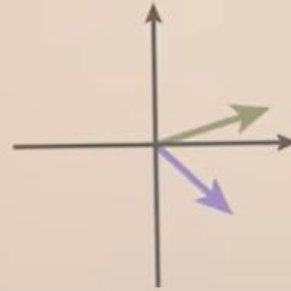




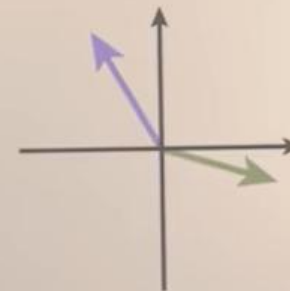
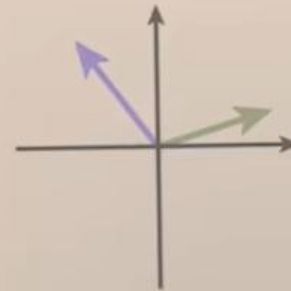
"A photo of a cat"



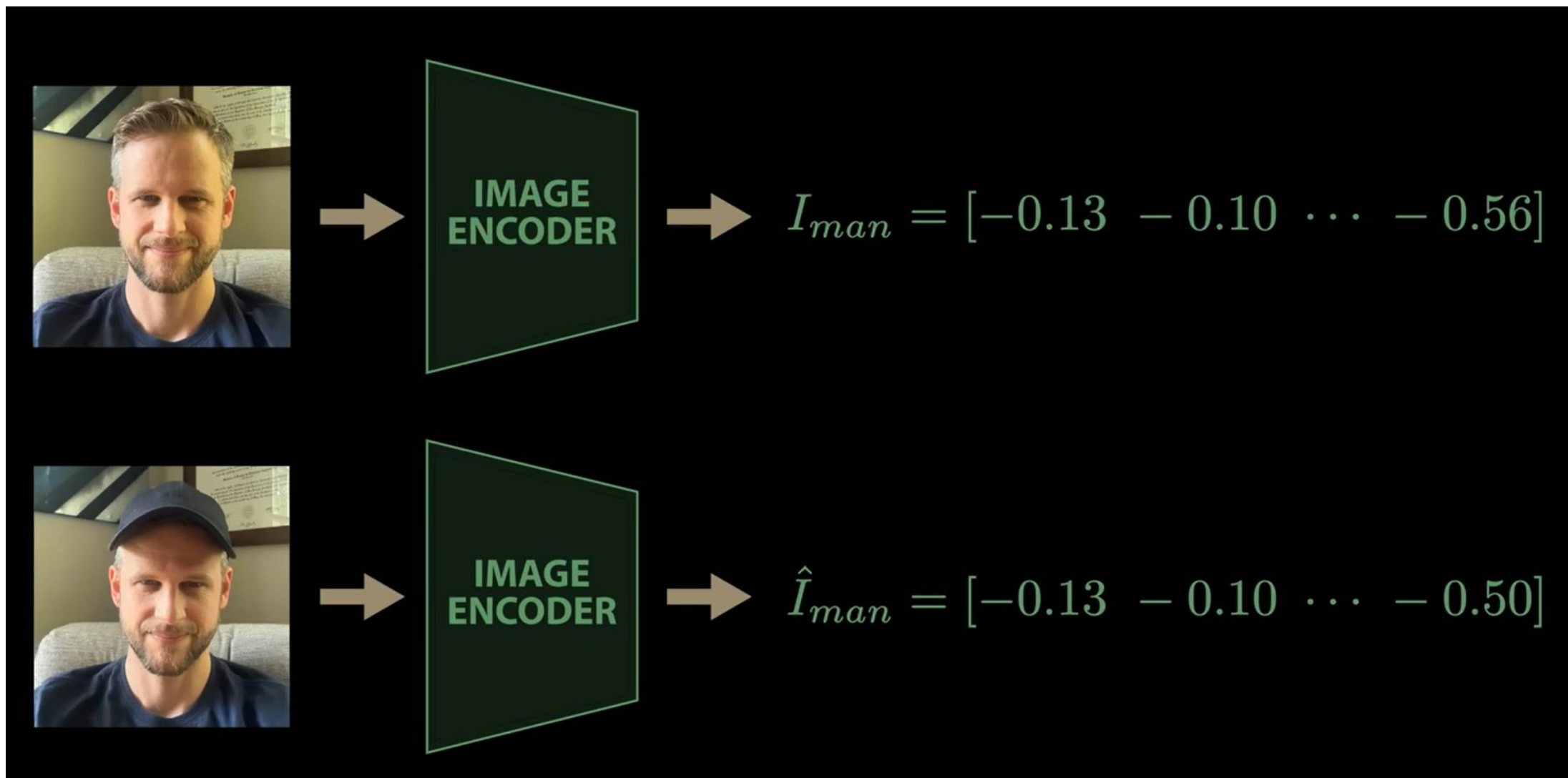
"A photo of a dog"



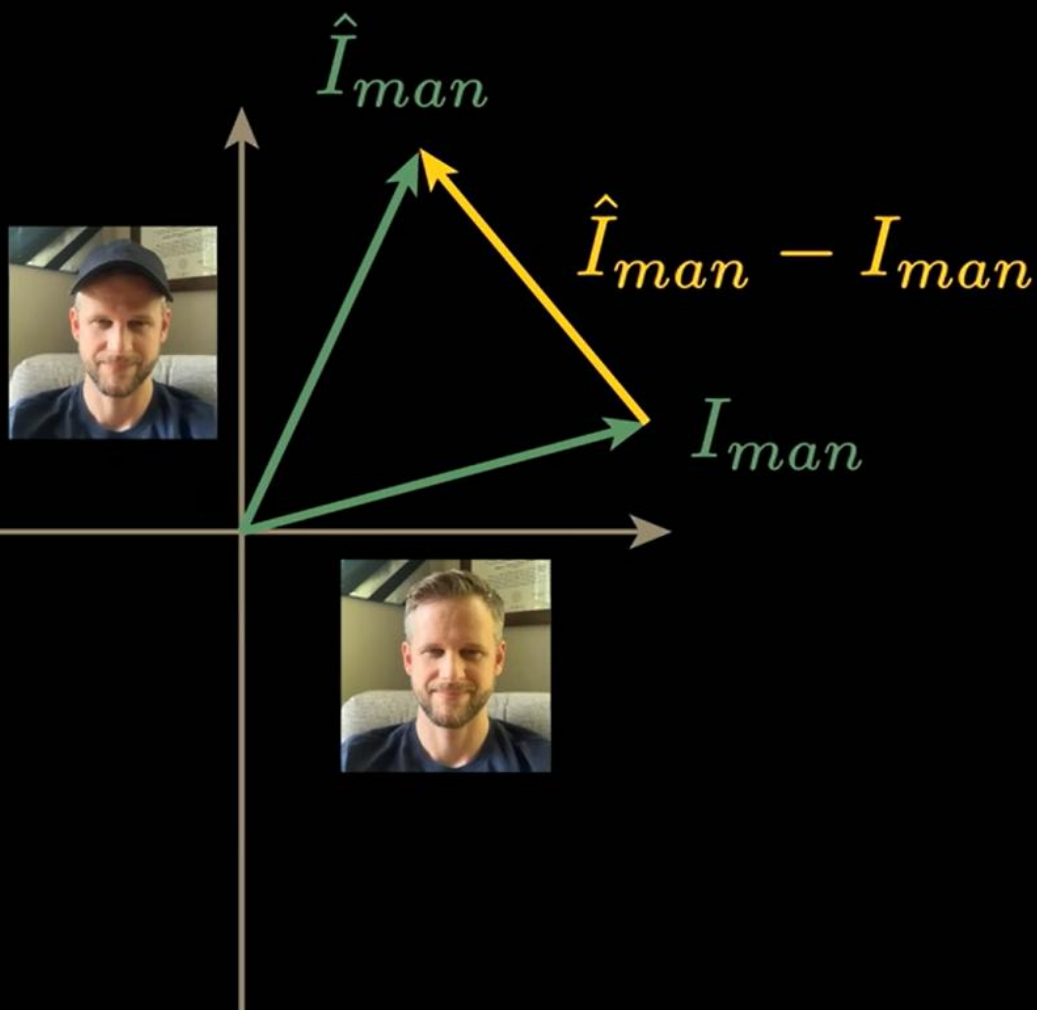
"A photo of a man"



CLIP learns semantic representation



CLIP learns semantic representation

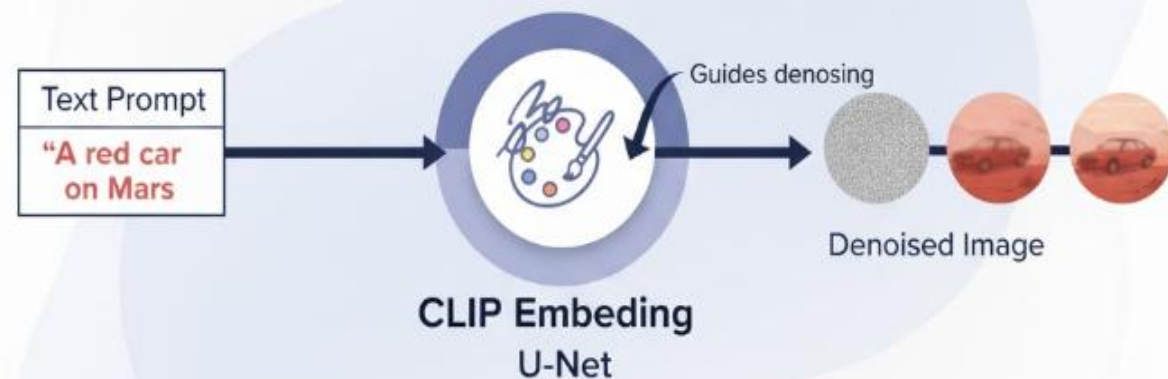


TOP MATCHES	
hat	0.165
cap	0.113
helmet	0.106
angry	0.061
exercising	0.06

Conditioning the Process with CLIP

- Text prompt → encoded as a CLIP vector in shared idea space.
- This vector conditions the U-Net at each denoising step.
- The model removes noise not randomly, but toward that concept.

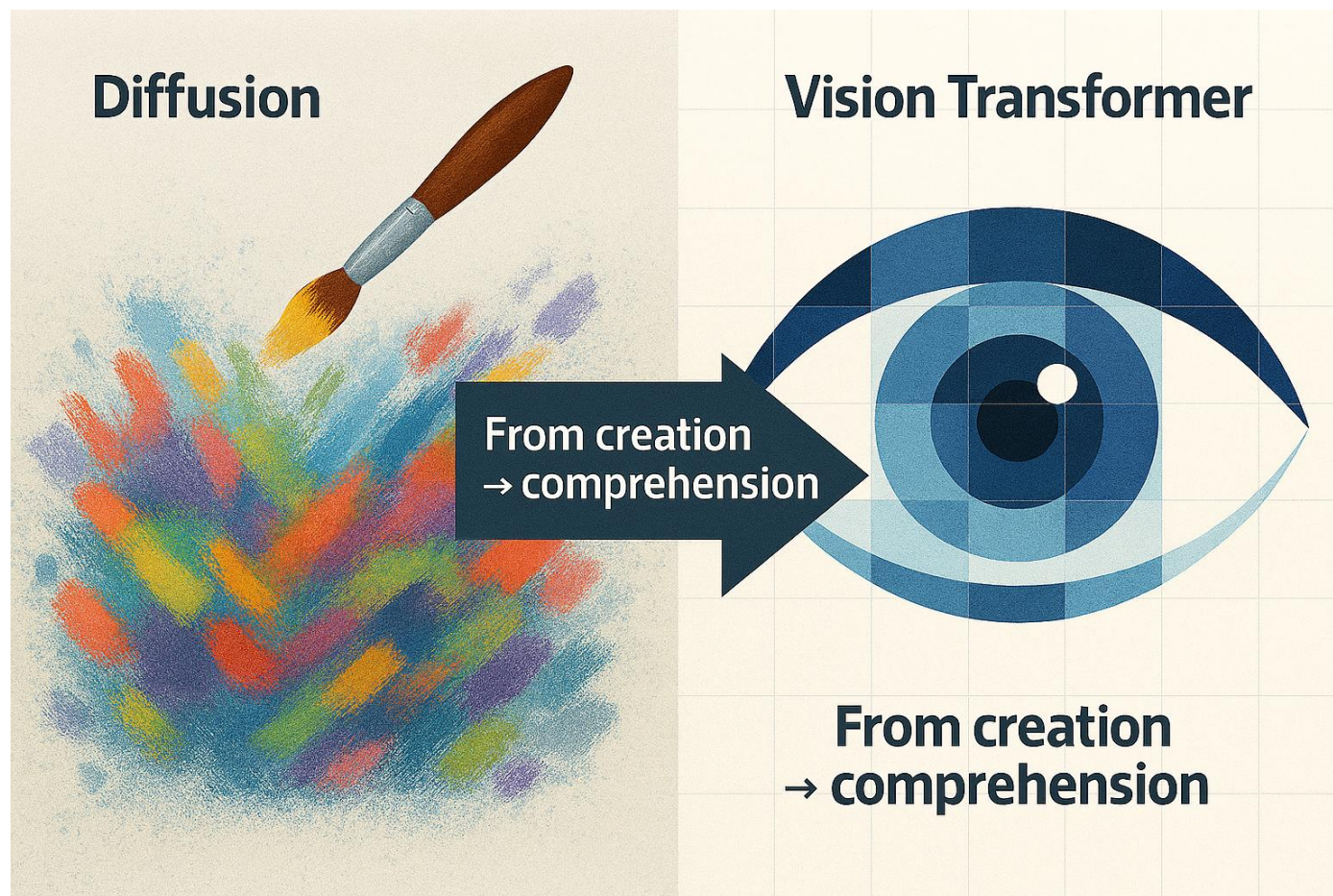
CLIP Gives Diffusion a Sense of Direction



Vision transformers

From Pixels to Perception

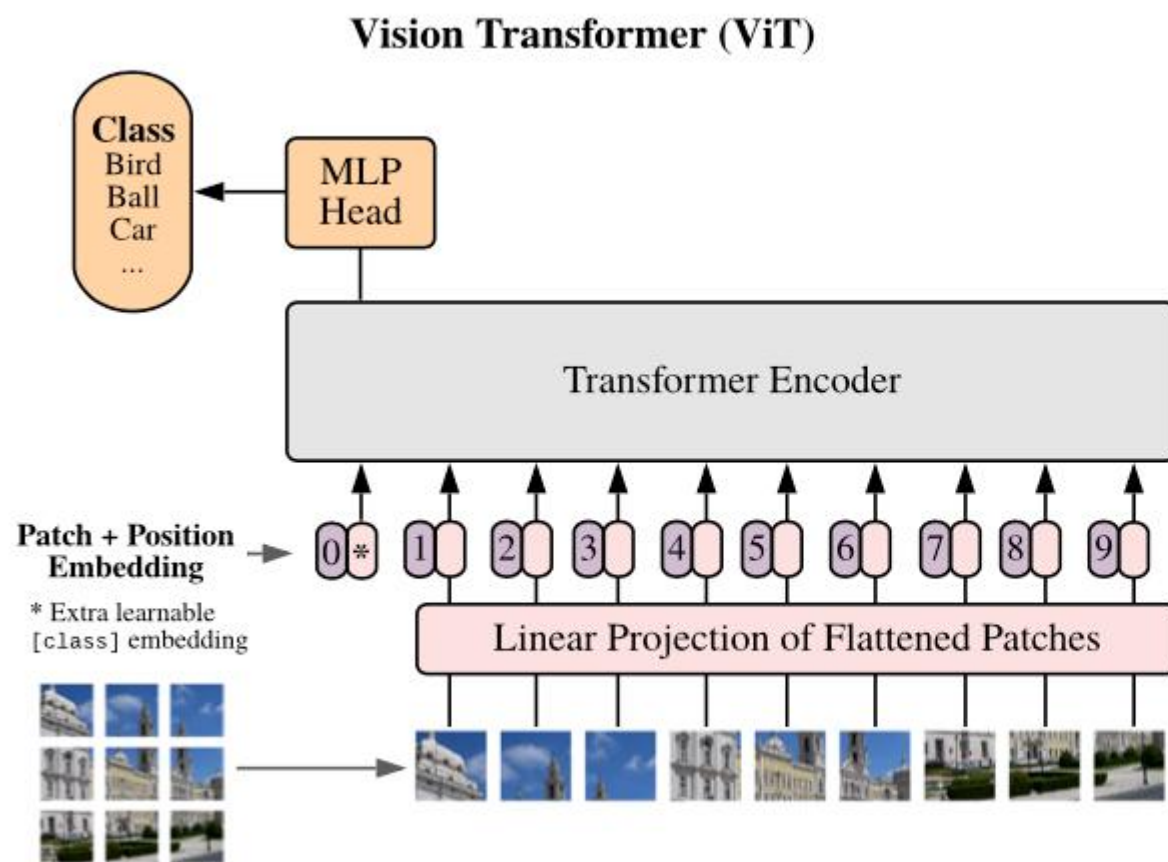
- Now that we can paint, can we use the same mechanism that allowed us to understand text to understand pictures as well?
- Can transformers mechanism be used to understand images?



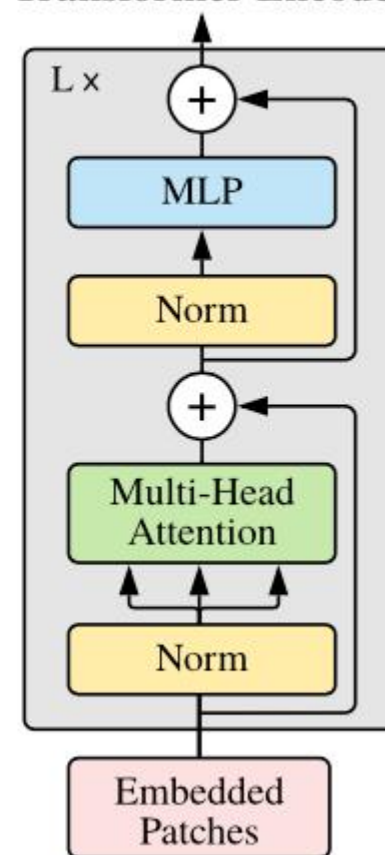
How Vision Transformers See

- CNN's looks at local pixels → great for edges, but not great for global context
- Vision transformers divide an image into small patches (16 x 16)
- Each patch becomes a “token”, just like a word in a sentence
- Self-attention lets the model understand relationships between these tokens.
- Allows it to relate distant parts of the image (“Dog and the ball”)

How Vision Transformers See (contd.)

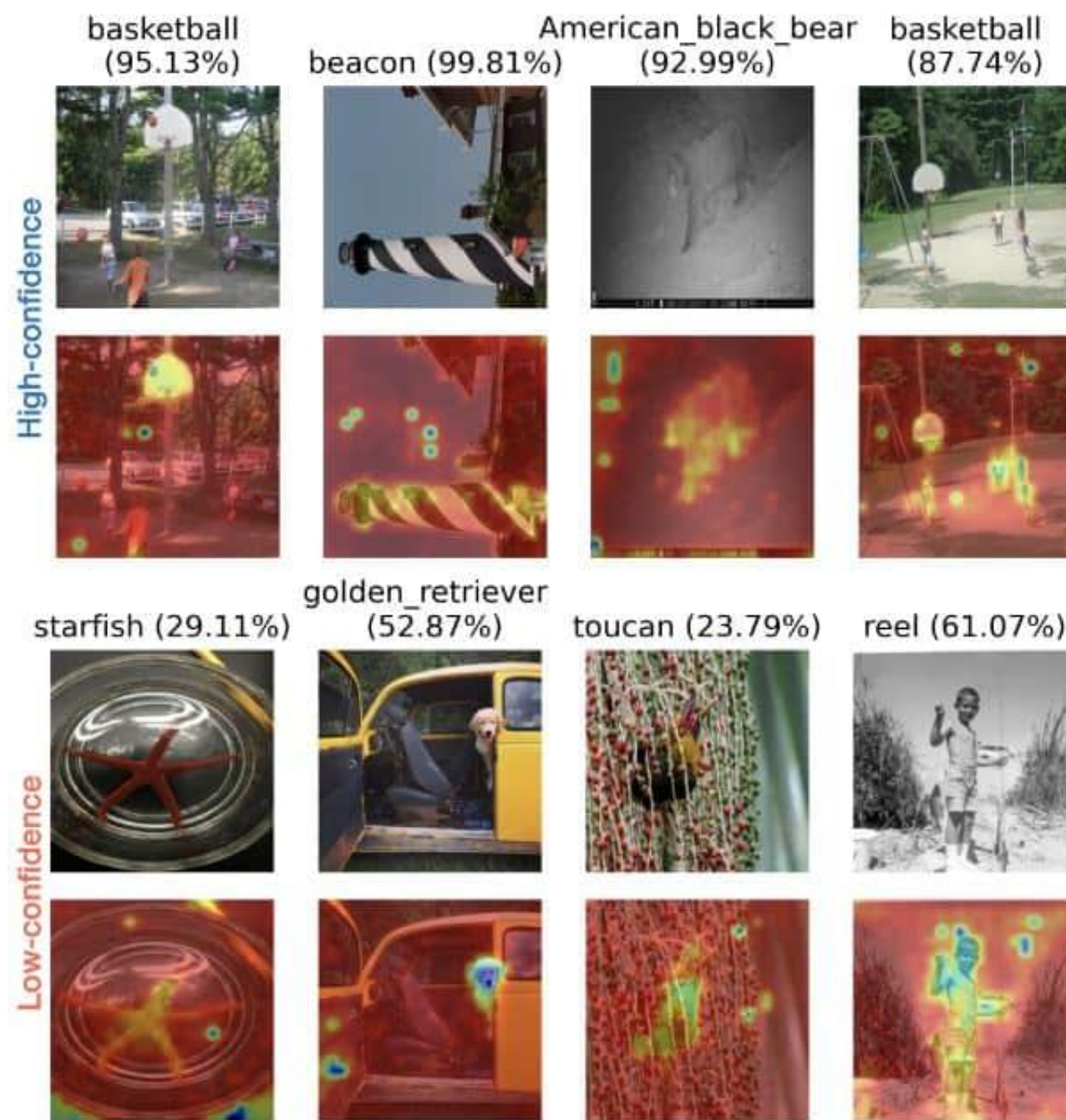


Transformer Encoder



CNNs were like looking through a keyhole - sharp locally but missing the whole scene.

Vision Transformers read the image like text (patch by patch) and use attention to connect the story.



Visualization of attention maps in VIT from: <https://viso.ai/deep-learning/vision-transformer-vit/>

Image Captioning & Cross-Modal Understanding

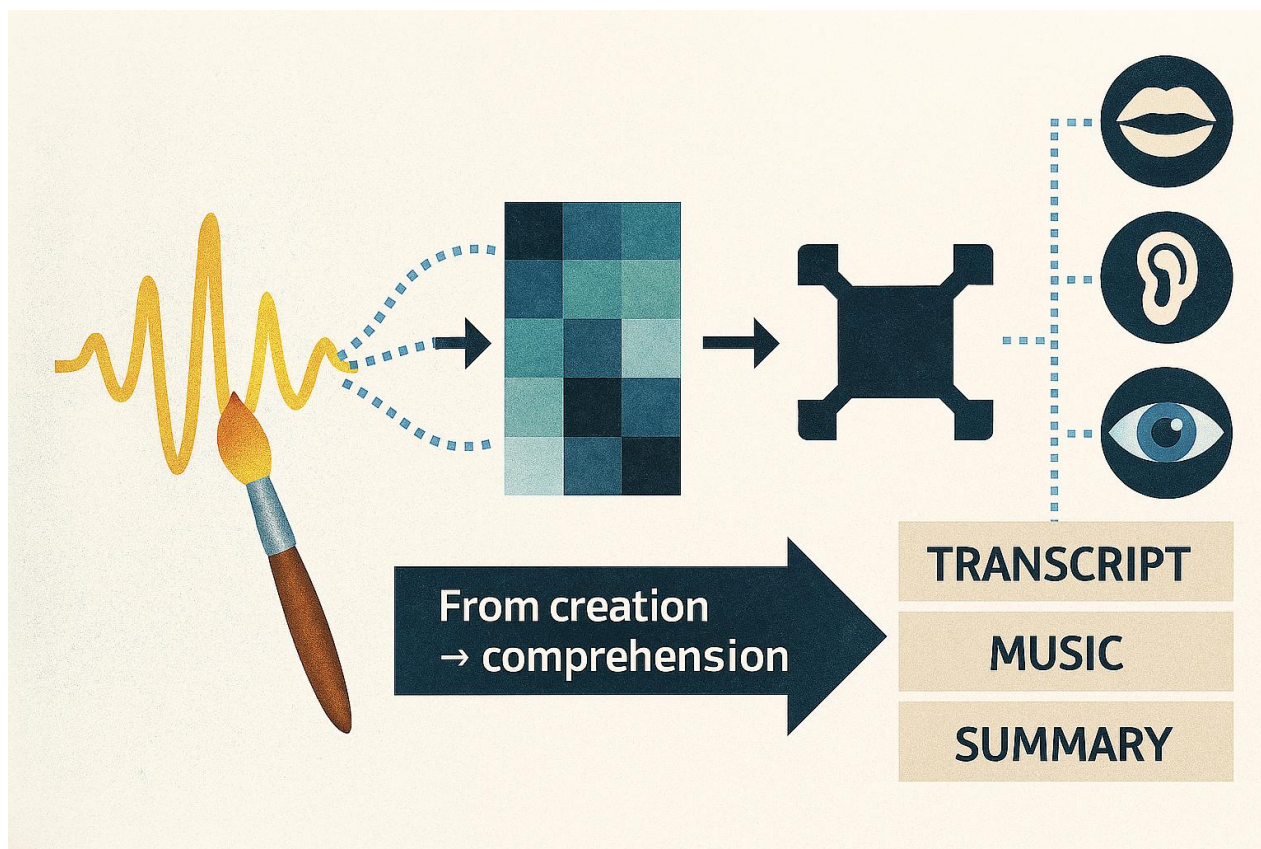
- Combine a Vision Transformer encoder + Language decoder → models that describe images in words.
- Early examples: BLIP, Flamingo (Google DeepMind).
- Used for search, accessibility, creative briefing, and context-aware agents.
- Vision and Language started sharing a common Transformer brain

From: <https://www.kaggle.com/code/burhanuddinlatsaheb/image-captioning-vit-gpt2>

A young boy is playing basketball. 	Two dogs play in the grass. 
A group of people walking down a street. 	A group of women dressed in formal attire. 
A skier is skiing down a snowy hill. 	A little girl in a pink shirt is swinging. 

Audio Transformers – Giving AI Ears

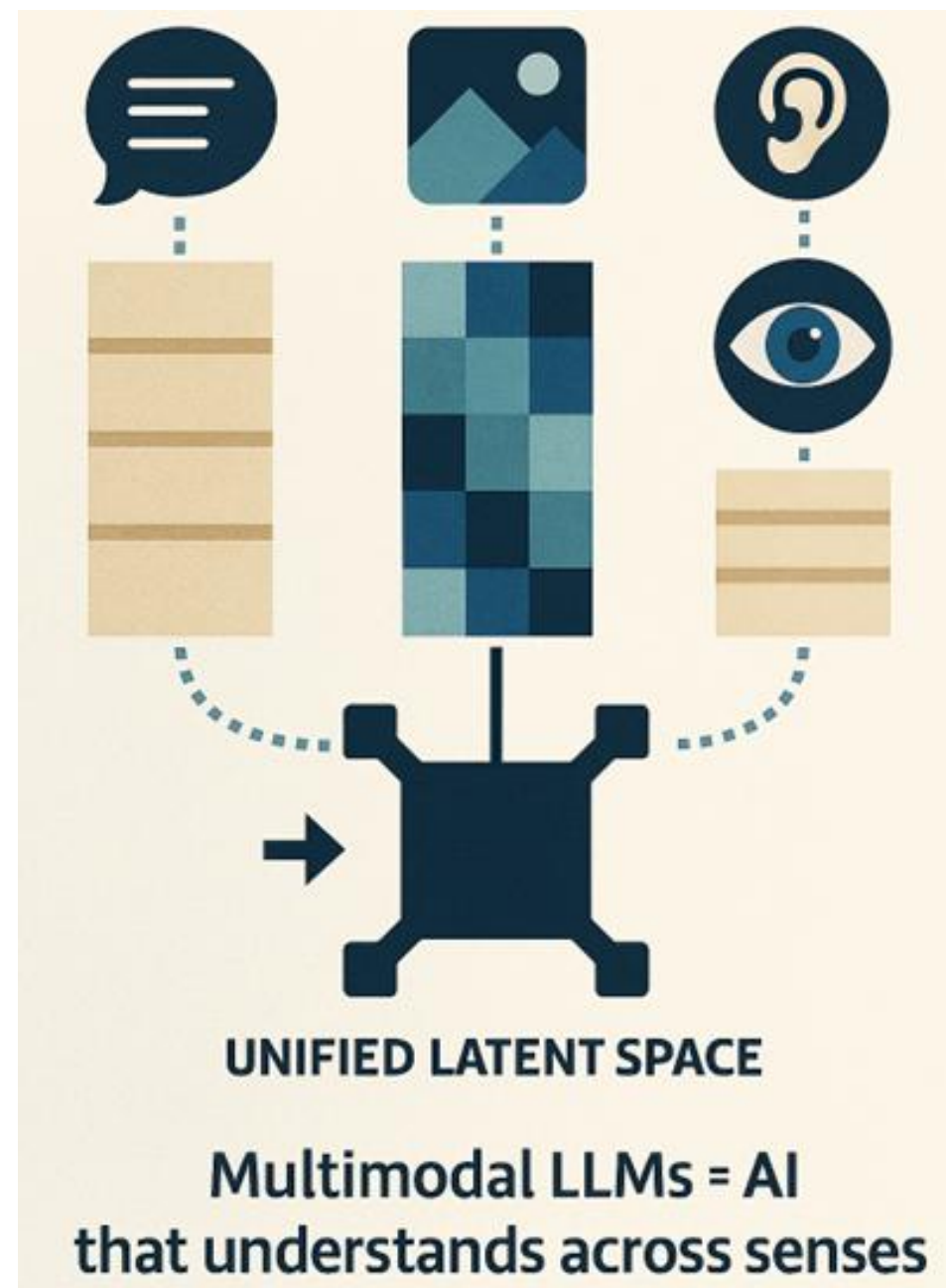
- Audio is just another sequence — a waveform or spectrogram of tokens.
- Models like Whisper, AudioLM, MusicGen apply Transformers to sound.
- This completes the triad of modalities: Text, Image & Audio



Transformers turned out to be universal; if you can tokenize it, they can learn it.

The Rise of Multimodal LLMs

- Modern LLMs (GPT-4o, Gemini, Claude 3, LLaVA) integrate multiple encoders into one shared transformer
- They reason across modalities (“*How many people are in this photo?*”)
- Enables true contextual AI agents which can read docs, see dashboards and listen to audio



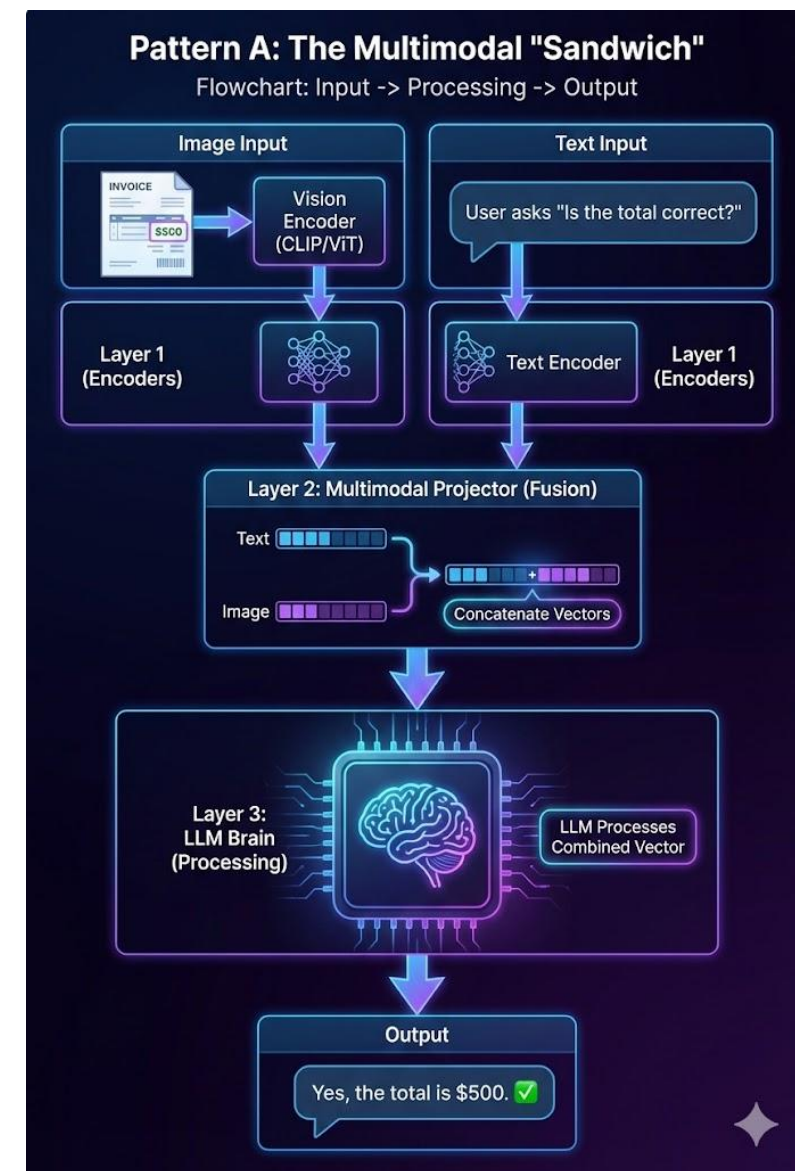
Multimodal, Multilingual LLMs

- Multimodal, multilingual LLMs are advanced AI systems that simultaneously process multiple data types—text, images, audio, and video—across dozens of languages. These models, such as the open-source **Pangea**, enhance cross-cultural understanding by training on diverse datasets that include, for example, 39 languages and 6 million instructions. By integrating these capabilities, they provide more nuanced, accurate, and culturally relevant outputs compared to traditional, English-centric LLMs.

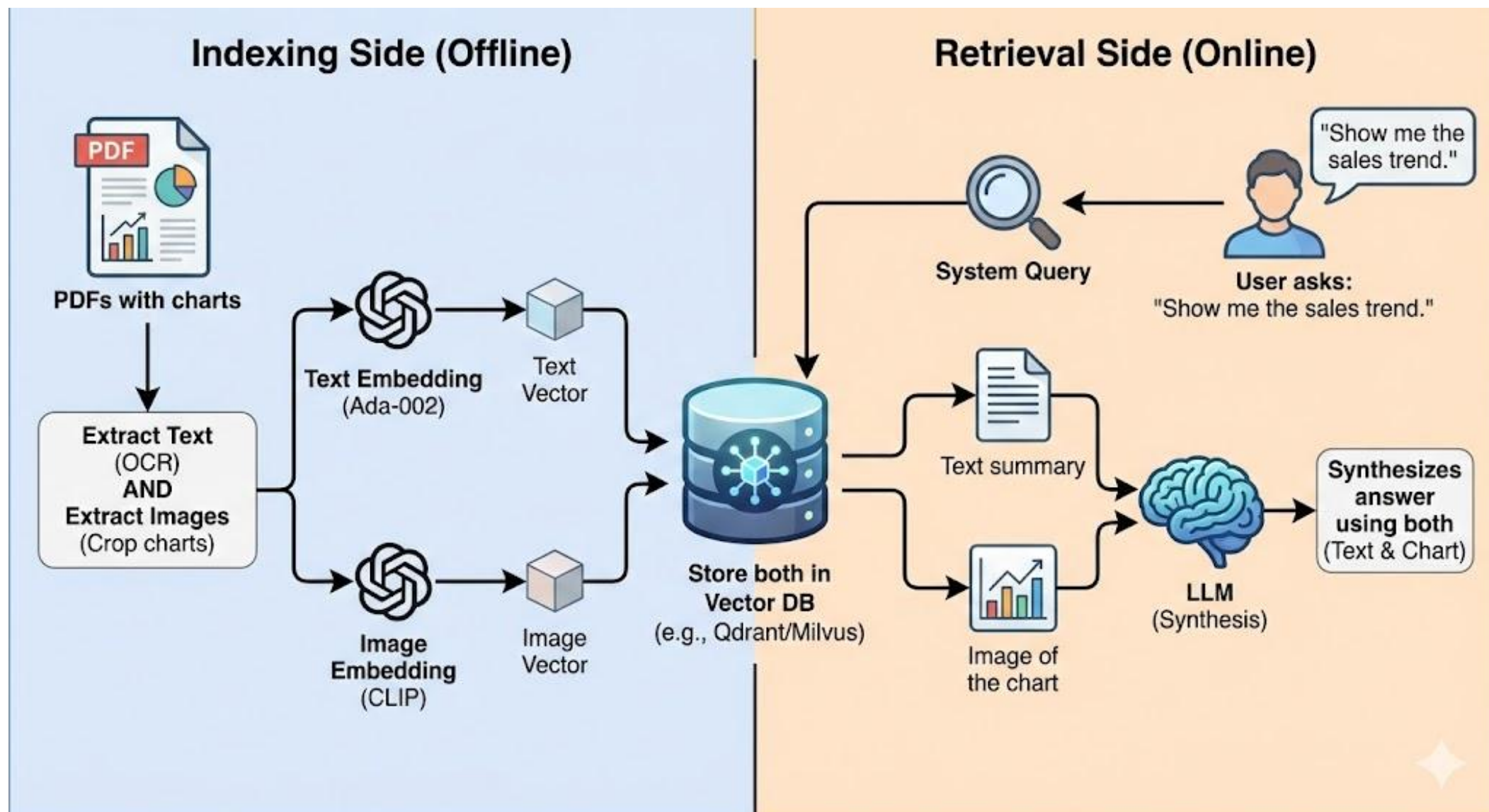
The Enterprise Architecture

The Canonical Multimodal pipeline

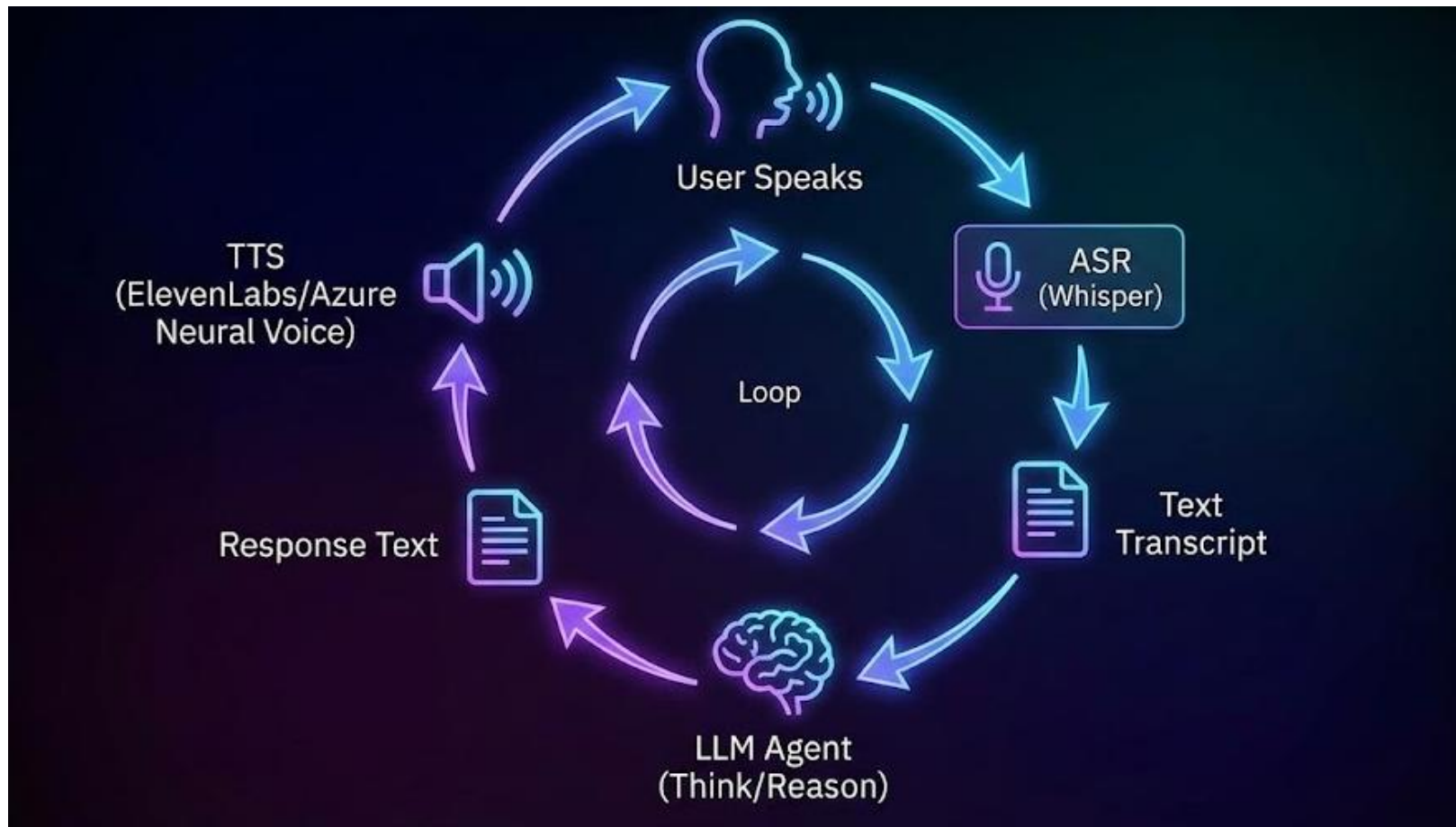
- Layer 1: Independent Encoders (**The senses**)
 - Text & Images are processed by specialized pre-trained encoders
 - Raw pixels → high dimensional feature vectors
- Layer 2: The multimodal projector (**The glue**)
 - The visual vectors & text vectors are combined
- Layer 3: The LLM (**The brain**)
 - LLM processes the combined sequence (image + text) in one pass
 - Cross-attention: the model links the text query to specific features in the image to answer questions



Pattern B: Multimodal RAG



Pattern C: Voice first flow



Engineering Constraints

- Voice activity detection: Knowing when the user stopped speaking is the one of the hardest parts.
- Latency Budget: You have ~800ms total for it to feel “conversational”.
- Streaming: You must stream the text from the LLM directly into the TTS engine chunk-by-chunk

Metrics

- Text Metrics:
 - BLEU/ROUGE (looks for exact word matches)
 - BERTScore/LLM-as-a-judge (Semantic matches)
- Multimodal Metrics:
 - Accuracy: Did it answer the question about the image correctly?
 - Hallucination Rate: Model claims “There is a red car” when there is none.
 - Clip Score: How well does the generated caption align with the image?
- For generative images, human evaluation is still the gold standard.

Summary: The Unified Model

- **Multilingual:** We solved language barriers by sharing concept embeddings, not just translating words
- **Multimodal:** We solve sensory barriers (vision/audio) by aligning them to the same text embedding space (CLIP)
- **Architecture:** Modern apps don't just generate text – they read documents, see charts and listen to voice using specialized encoders fused into an LLM.
- **Next :** Autonomous decision making with..

AGENTS



Thank You

We appreciate your time and look forward to your feedback on the session.

Contact us: TekFrameworks Consulting | info@tekframeworks.com | www.tekframeworks.com

Confidentiality Notice: This proposal is proprietary to **TekFrameworks Consulting Pvt. Ltd.** and is intended solely for the Client's evaluation. It may not be shared with third parties. The contents are non-binding; any engagement will be governed by a separate written agreement. For Client Use Only | TekFrameworks Consulting Pvt. Ltd. | © 2025 TekFrameworks